Walfy Admin Doc 2.6.0

22-03-21 Last updated

—

“It's an offline document. There is an online documentation here. We are highly recommending the online version. Because it will always be updated.”

Introduction

Welcome to Walfy documentation & thank you for buying our product. So there are two setup steps here. Admin panel & Walfy App. Both are developed on Flutter. So, You have to install flutter on your computer. You can use Both Visual Studio Code & Android Studio for Flutter. We have used Visual Studio Code. So, Our Setup will be based on this IDE.

To install flutter on your computer, follow the official documentation from Google or watch youtube tutorials.

Flutter Official Site : https://flutter.dev/.

Admin Panel Setup

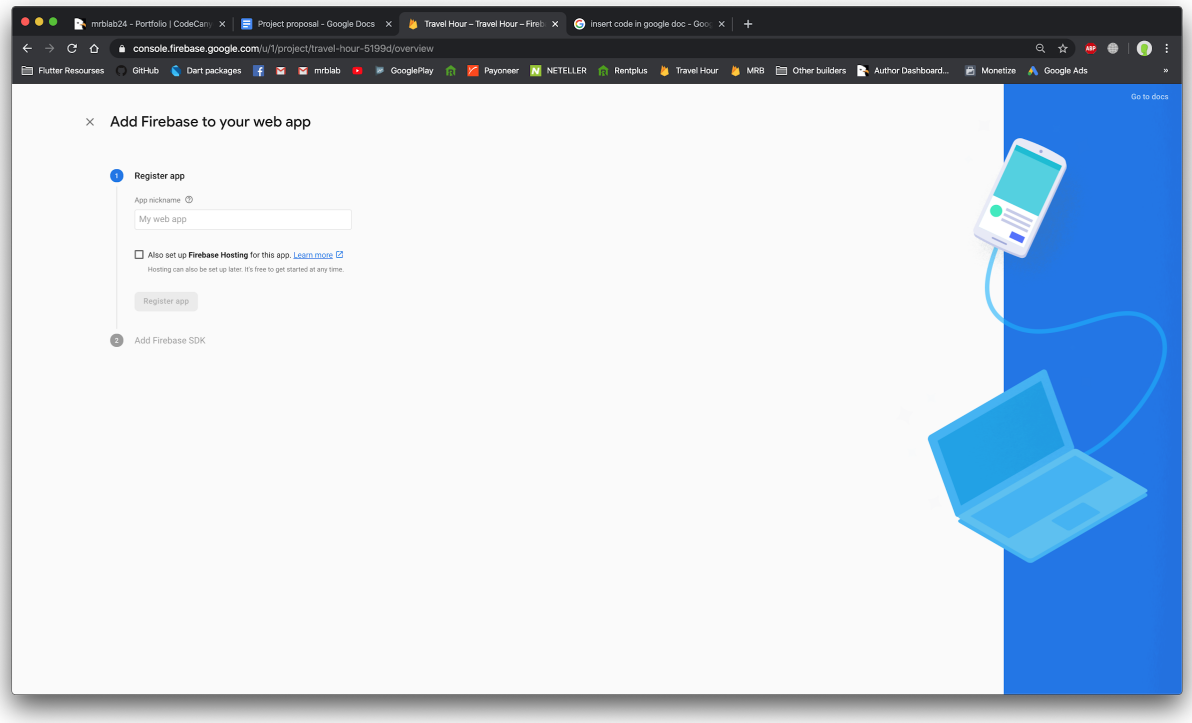
After successfully installing Flutter on your computer, You have to enable flutter web support. Make sure you have installed the stable channel. From Flutter stable version 2.0.0, it will support the web. So, If you are already on the latest stable channel of flutter, you don't have to do anything for the flutter web.

Just run the following code to enable flutter web support.

```
flutter config --enable-web
```

And then it will enable the web feature for your project.

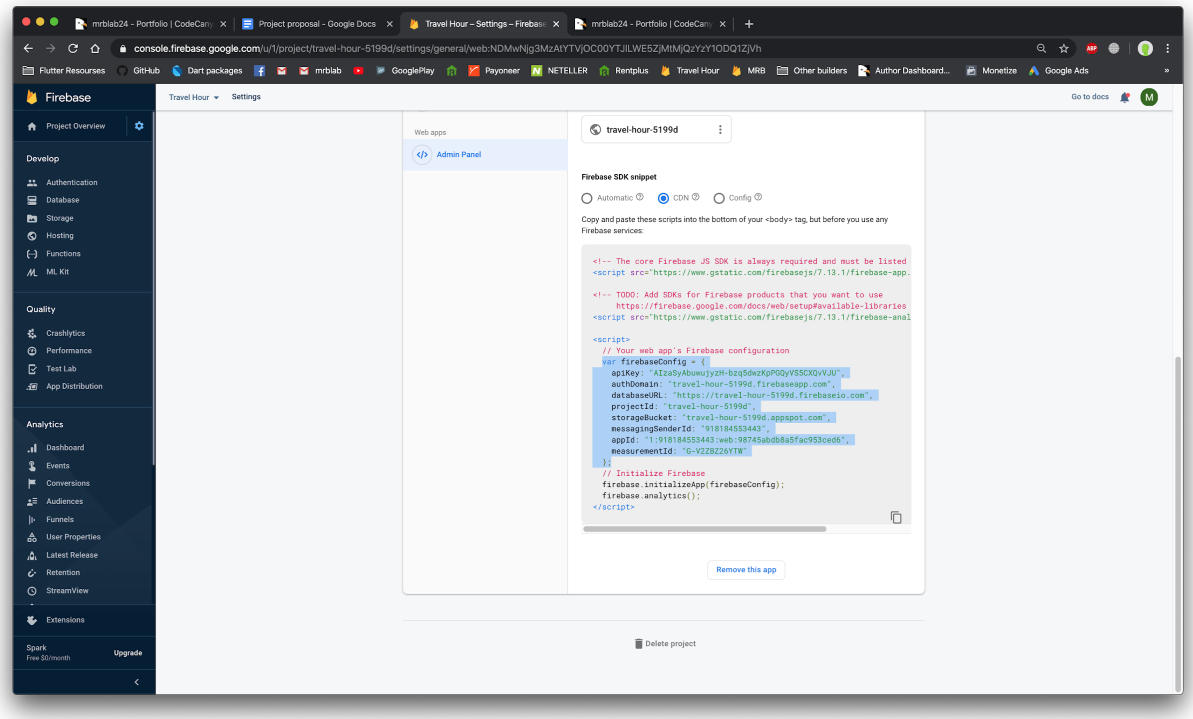
1. First, you need to set up the database. We used firestore database as backend. So, go to firebase console and create a project by your app name. And go to project overview and click on the **plus icon** and then click on **web**. You will see a popup like this :



2. Input the name like as Admin Panel and fill the checkbox because you need hosting too and then click on **register app**. After that, Skip the other options by clicking next.

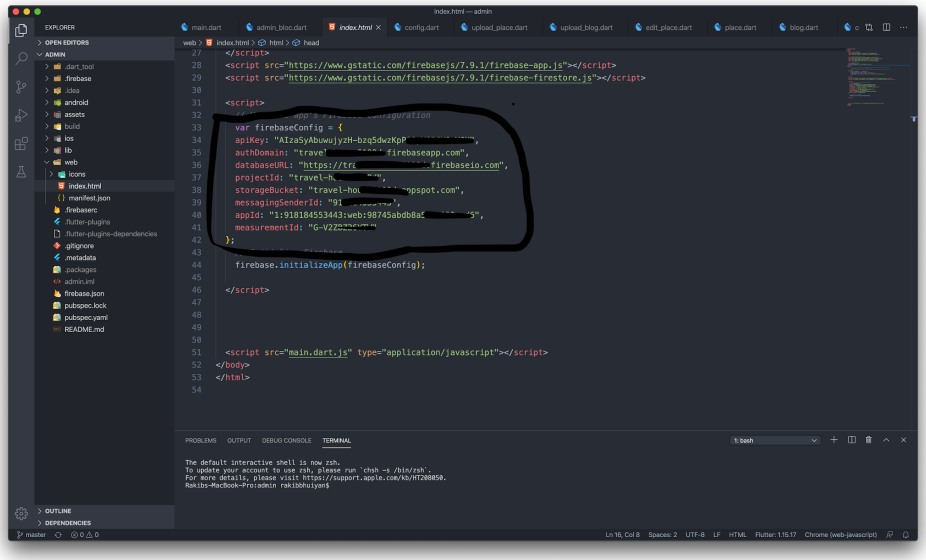
3. Now go to the Project settings and in the bottom section click on the **Admin panel** that you have just created.

4. After that click on **CDN** and copy the selected code which is shown in the picture below.

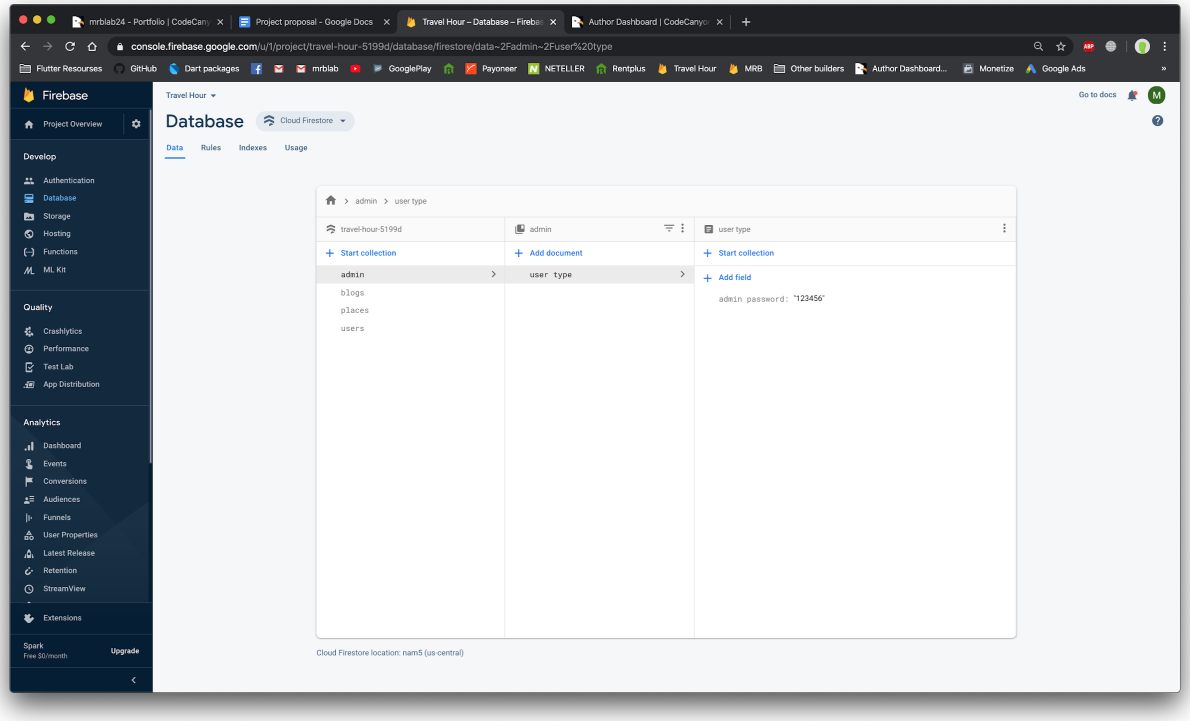


5. Now, go to the IDE VSCode or Android Studio and open the admin folder.
6. Go to this file and replace the selected code with your config code.

admin/web/index.html



7. Now go to the Project Overview page and click on database and create a firestore database as test mode.
8. Now, Create a collection by “admin” and create a document by ”user type’ and create a textfield by “**admin password**”. Just like the picture below. Only the admin section. And then Change the password.



9. Now go to,

admin/lib/models/config.dart

```
class Config{
  final String appName = 'Walfy';   //app name
  final String testerPassword = 'tester';   //testing password
}
```

10. Now, Edit your App Name by replacing **Walfy**.
11. So, Your Admin app is complete. You can run by typing this command.

```
flutter run -d chrome
```

12. Now you need to upload your site to your firebase hosting. For this, you need to install firebase tools on your IDE. Run this command on the terminal:

    It will download firebase tools for you. Or you directly download from here and install this manually.

```
npm install -g firebase-tools
```

13. After that, You need to login to firebase console with your google account. To do that, Run:

```
firebase login
```

    Follow the command prompt and the link in your browser. Accept the permissions by clicking Ok. When you return to your terminal you should see that you are now logged in:

```
Waiting for authentication...

✔  Success! Logged in as [redacted]
antrob-macbookpro3:flutter_web_test antrob$
```

14. Now you need to initialize firebase. Run this command:

```
firebase init
```

```
antrob-macbookpro3:flutter_web_test antrob$ firebase init

     ######## #### ########  ######## ########     ###     ######  ########
     ##       ##  ##    ## ##       ##     ##  ##   ##  ##      ##
     ######   ##  ########  ######   ########  #########  ######  ######
     ##       ##  ##    ##  ##       ##     ## ##     ##      ## ##
     ##       #### ##    ## ######## ########  ##     ##  ######  ########

You're about to initialize a Firebase project in this directory:

  /Users/antrob/StudioProjects/flutter_web_test

? Which Firebase CLI features do you want to set up for this folder? Press Space to select features, then Enter to confirm your choices. (Press <space> to select, <a> to toggle all, <i> to invert selection)
 ○ Database: Deploy Firebase Realtime Database Rules
 ○ Firestore: Deploy rules and create indexes for Firestore
 ○ Functions: Configure and deploy Cloud Functions
>◉ Hosting: Configure and deploy Firebase Hosting sites
 ○ Storage: Deploy Cloud Storage security rules
```

15. Use the arrow keys to navigate the cursor to **Hosting** and hit the spacebar to select it, then press enter. Now you will see this screen:

```
You're about to initialize a Firebase project in this directory:

  /Users/antrob/StudioProjects/flutter_web_test

? Which Firebase CLI features do you want to set up for this folder? Press Space to select features, then Enter to confirm your choices. Hosting: Configure and deploy Firebase Hosting sites

=== Project Setup

First, let's associate this project directory with a Firebase project.
You can create multiple project aliases by running firebase use --add,
but for now we'll just set up a default project.

? Please select an option: (Use arrow keys)
> Use an existing project
  Create a new project
  Add Firebase to an existing Google Cloud Platform project
  Don't set up a default project
```

Select **Use an existing project** by pressing Enter. Use the arrow keys to select the project you have made in the firebase console.

16. Next, enter **build/web** as the public directory and press enter, then enter **y** (for yes) to select the single page app option:

```
Your public directory is the folder (relative to your project directory) that
will contain Hosting assets to be uploaded with firebase deploy. If you
have a build process for your assets, use your build's output directory.

? What do you want to use as your public directory? build/web
? Configure as a single-page app (rewrite all urls to /index.html)? (y/N) y
```

17. Now Run this command to build the web app.

```
Flutter build web
```

18. After that run this command to upload the app to your hosting.

```
firebase deploy
```

That's it. Your Admin Panel is 100% complete. You will get a url from terminal and it will be the domain of your admin panel. **Before setting up the app, upload at least 1 category and 6-10 contents from the admin panel.**

Now go for the app setup.....!

Bugs & Fixes (Only for V2.5.0):

1. **List deprecation error :** You could see an error like this because of the flutter recent update.

```
final FirebaseFirestore firestore = FirebaseFirestore.instance;
ScrollController controller;
DocumentSnapshot _lastVisible;
bool _isLoading;
List<DocumentSnapshot> _snap = new List<DocumentSnapshot>();
List<StateModel> _data = [];
final scaffoldKey = GlobalKey<ScaffoldState>();
final String collectionName = 'states';
```

To fix this, just remove this :

```
new List<DocumentSnapshot>()
```

and add this :

```
[]
```

2. **FlatButton Deprecation Error (Without color & shape) :**

```
),  // TextButton
SizedBox(width: 10),
FlatButton(
  child: Text(
    'No',
    style: TextStyle(
        color: Colors.white,
        fontSize: 16,
        fontWeight: FontWeight.w600), // TextStyle
  ), // Text
  onPressed: () => Navigator.pop(context),
), // FlatButton
```

To fix this error, just rename **FlatButton** by **TextButton**.

3. **FlatButton with color & shape Error** : If the deprecated FlatButton has color & shape like the picture below :

```
FlatButton(
  color: Colors.redAccent,
  shape: RoundedRectangleBorder(
    borderRadius: BorderRadius.circular(3)
  ), // RoundedRectangleBorder
  child: Text(
    'No',
    style: TextStyle(
        color: Colors.white,
        fontSize: 16,
        fontWeight: FontWeight.w600), // TextStyle
  ), // Text
```

Then first rename **FlatButton** to **TextButton** and then remove the **color & shape** parameter with value and also **remember the color & radius.**

Now add some code like this :

```
TextButton(
  style: ButtonStyle(
    padding: MaterialStateProperty.resolveWith((states) => EdgeInsets.only(left: 40, right: 40, top: 15, bottom: 15)),
    backgroundColor: MaterialStateProperty.resolveWith((states) => Colors.deepPurpleAccent),
    shape: MaterialStateProperty.resolveWith((states) => RoundedRectangleBorder(borderRadius: BorderRadius.circular(25))),
  ), // ButtonStyle
  child: Text(
    'No',
    style: TextStyle(
        color: Colors.white,
        fontSize: 16,
        fontWeight: FontWeight.w600), // TextStyle
  ), // Text
```

Make sure you have replaced the old color & shape value. That's it.